

Orli - A new take on Print-on-Demand

Project Team

Anishra Ashfaq	22I-2614
Muhammad Uzair Choudhary	22I-2681
Aisha Adil	22I-2718

Session 2022-2026

Supervised by

Dr. Behjat Zuhaira



Department of Software Engineering

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Scope	2
1.3	Modules	3
1.3.1	Module 01: Client Web Application (React Frontend)	3
1.3.1.1	Customer Interface	4
1.3.1.2	Designer Interface	4
1.3.1.3	Manufacturer Interface	4
1.3.2	Module 02: Admin Web Application (React + Node.js)	4
1.3.2.1	Admin Interface	5
1.3.3	Module 03: AI Services (Flask + TensorFlow)	5
1.3.3.1	AI and Creative Intelligence Module	5
1.3.4	Module 04: Data Management Services (MongoDB-tentative)	5
1.3.4.1	Data Management Module	6
1.3.5	Module 05: Core Server Backend and Financial Operations (Node.js + Express + Payment Gateway)	6
1.3.5.1	Core Business Logic Module	6
1.3.5.2	Financial Operations Module	6
1.3.6	System Overview	7
1.4	User Classes and Characteristics	7
1.4.1	Customer (End User/ Buyer)	7
1.4.1.1	Description	7
1.4.1.2	Characteristics	8
1.4.1.3	Primary Goals	8
1.4.2	Designer (Content Creator/ Seller)	8
1.4.2.1	Description	8
1.4.2.2	Characteristics	8
1.4.2.3	Primary Goals	9
1.4.3	Manufacturer (Production Partner/ Vendor)	9
1.4.3.1	Description	9
1.4.3.2	Characteristics	9
1.4.3.3	Primary Goals	9

1.4.4	Administrator (System Operator/ Platform Manager)	10
1.4.4.1	Description	10
1.4.4.2	Characteristics	10
1.4.4.3	Primary Goals	10
2	Project Requirements	13
2.1	Use-cases	13
2.1.1	Identifying System Boundaries, Actors and their Goals	13
2.1.2	Use Cases	17
2.1.2.1	Use Case 1: Browse and Search Products	17
2.1.2.2	Use Case 2: Place Order	19
2.1.2.3	Use Case 3: Create and Enhance Design Using AI	21
2.1.2.4	Use Case 4: Manage Storefront	22
2.1.2.5	Use Case 5: Fulfill Order (Manufacturer)	23
2.1.2.6	Use Case 6: Moderate Platform Content	23
2.1.3	UML Use Case Diagram	24
2.2	Functional Requirements	25
2.2.1	Module 1: Client Web Application (React Frontend)	25
2.2.1.1	Customer Interface	25
2.2.1.2	Designer Interface	26
2.2.1.3	Manufacturer Interface	27
2.2.1.4	Admin Interface	28
2.2.2	Module 2: AI and Creative Intelligence (Flask + TensorFlow)	29
2.2.3	Module 3: Core Business Logic (Node.js + Express)	30
2.2.4	Module 4: Data Management (MongoDB)	31
2.2.5	Module 5: Financial Operations (Payment Gateway Integration)	31
2.3	Non-Functional Requirements	31
2.3.1	Reliability	32
2.3.2	Usability	32
2.3.3	Performance	32
2.3.4	Security	32
2.4	Domain Model	32
3	System Overview	35
3.1	Architectural Design	35
3.2	Design Models	37
3.3	Data Design	45
3.3.0.1	Data Transformation	46
3.3.0.2	Major Entities	46
3.3.0.3	Data Storage Systems	46
3.3.0.4	Processing and Organization	46

CONTENTS

3.3.0.5 Summary 46

References 47

List of Figures

2.1	UML Use-Case Diagram	24
2.2	UML Domain Model	33
3.1	System Architecture	36
3.2	System Architecture from another View	37
3.3	Class Diagram	38
3.4	Activity Diagram for Designers	39
3.5	Activity Diagram for Designers	40
3.6	Activity Diagram for Designers	41
3.7	Activity Diagrams for Customers	42
3.8	Sequence Diagram for Customer for Order Placement	43
3.9	System Sequence Diagram for Order Placement	44
3.10	System Sequence Diagram for Designer Usecases	45

List of Tables

1.1	User Classes, Interfaces, and System Interaction Characteristics for Orli .	11
2.1	Actor Summary	14
2.2	Actor–Goal (Use Case) Mapping and Validation for Orli	15

Chapter 1

Introduction

This project report presents the detailed requirements specification for Orli, an innovative Print-on-Demand (PoD) digital platform designed to empower local artists, independent designers, and small-scale entrepreneurs. The application bridges the gap between creators, manufacturers, and consumers through an integrated, AI-enhanced e-commerce ecosystem. Built on the MERN stack with Python-based AI modules, Orli reimagines the traditional print-on-demand model by combining creative autonomy, technological intelligence, and a marketplace tailored for the Pakistani context.

Orli enables designers to upload, generate, and enhance artwork using AI-powered tools, manage personalized storefronts, and connect with manufacturers for order fulfillment. Customers can browse, preview, and purchase these designs in real time, while administrators and manufacturers manage logistics, analytics, and quality assurance.

This document defines the system requirements, architecture, design goals, and constraints of Orli. It aims to prevent misinterpretations during the development life cycle and to guide decision-making by providing a clear, structured framework for implementation.

This document is intended for multiple types of readers, each with distinct objectives and technical backgrounds. The readers may be:

- **Project Team** (us): To understand system architecture, APIs, module interactions, and technical dependencies required for implementation, oversee progress, manage timelines and resources, identify functional requirements, design test cases, and validate that the final product meets all specified criteria, and maintain manuals and documentation.
- **Users** (Designers, Manufacturers, and Customers): To comprehend the system's purpose, its role in enhancing creative entrepreneurship, and the expected user experience.
- **Academic Evaluators and Supervisor**: To assess the project's clarity, technical scope, and alignment with academic standards.

Through this report, the team plans to establish a clear foundation for Orli's development, ensuring that creativity, technology, and commerce converge to foster sustainable digital entrepreneurship for emerging artists and small businesses.

1.1 Problem Statement

In the evolving landscape of the print-on-demand industry, there is a notable absence of an integrated platform that bridges the gap between local talent and global markets. Existing solutions fall short in offering advanced AI-powered design tools and a marketplace that caters to both local and international demands. This gap creates challenges for local artists to showcase their work on a global stage and limits customers' access to unique and culturally rich designs. It's a missed opportunity for both creators and consumers.

Expanding further, this gap limits the growth of independent designers and weakens the customer experience. Our platform addresses this by introducing a localized POD model tailored for emerging artists and entrepreneurs. Beyond simplifying production and fulfillment, it empowers creators to build and manage their own brands, laying the foundation for sustainable digital entrepreneurship in Pakistan.

1.2 Scope

By bridging the gap between manufacturers, creative designers, and consumers, our proposed platform fosters a collaborative and decentralized approach to product creation, customization, and commerce. It will serve as a comprehensive digital ecosystem for artists and designers, offering a range of core functionalities tailored to support creative entrepreneurship and localized print-on-demand services. Designers/Users will be able to create personalized profiles, upload original designs, create designs on our canvas, and utilize AI-assisted tools for design generation and enhancement. This would be powered by three AI models each for design generation, enhancement, and mock-up context generation. The system will include product customization features **in terms of designs** (on **t-shirts** and **stickers**, currently) with real-time previews, enabling creators to visualize their work on various merchandise formats. A dedicated manufacturer dashboard will facilitate order tracking and production coordination, while integrated analytics tools will provide insights into sales performance and customer engagement. The platform will also host a curated marketplace for browsing and purchasing the designed products, supported by SEO-optimized e-commerce capabilities on product pages to enhance discoverability. Importantly, the scope excludes anything not explicitly mentioned, advanced marketing automation, and enterprise-level integrations—ensuring the solution remains focused, technically feasible, and aligned with the current resources and expertise of the team.

We have intentionally made these exclusions to ensure that the project remains focused, achievable, and aligned with the current capabilities and resources of the team.

1.3 Modules

In its most rudimentary form, our app is an e-commerce platform. But on a closer look, it's a web-application that re-imagines the traditional online marketplace by merging the core functionalities of platforms like Printify, Shopify, and Etsy into a unified e-commerce ecosystem. Unlike conventional e-commerce models in Pakistan, our platform serves as a dynamic, multi-sided marketplace that connects three key stakeholders: manufacturers, independent designers, and end consumers.

At its core, the platform empowers designers to build their own brands and storefronts by sourcing materials directly from verified print providers. Designers can create and showcase custom products using integrated design tools, including Adobe-style or Canva-style interfaces and AI-powered modeling features. These tools allow for real-time 3D-rendering enabling designers and customers to visualize products in their environment before publishing/purchasing.

To ensure clarity in architecture and role-based functionality, the system is divided into five major modules.

1. Client Web Application
2. Admin Web Application
3. AI Services (Intelligent Assistance Layer)
4. Data Management Services
5. Core Server Backend & Financial Operations

Each system type contains relevant modules and sub-modules that interact through RESTful APIs and shared data pipelines. Below is the architectural visualization of our system focusing on each module separately to ensure all the details are up-front.

1.3.1 Module 01: Client Web Application (React Frontend)

This module represents all user-facing interactions, primarily designed using **React.js** for responsive, modular, and component-driven interfaces. It includes three major user roles—**Customer**, **Designer**, and **Manufacturer**—each with its own dedicated UI module.

1.3.1.1 Customer Interface

It handles all front-end functionalities for end-users who browse, preview, and purchase products. Features include:

1. Preview Products
2. Checkout and Track Orders
3. View Trends
4. Browse Products

1.3.1.2 Designer Interface

This interface empowers designers to create, manage, and sell custom merchandise. Features include:

1. Design Canvas (Canva/Adobe-like interface)
2. Storefront Creation and Management
3. Order Management (may be changed on implementation)
4. Trend Analytics
5. Product Page SEO-Optimization

1.3.1.3 Manufacturer Interface

It caters to verified production/print partners who shall handle printing, inventory, and shipping. Features include:

1. Manage Orders (accept/reject/defer)
2. Print and Ship
3. Manage Inventory
4. Manage Performance Statistics

1.3.2 Module 02: Admin Web Application (React + Node.js)

This is a dedicated administrative dashboard providing complete system control, monitoring, and moderation. Its features are as follows:

1.3.2.1 Admin Interface

1. User Management
2. System Monitoring
3. Content Moderation
4. Analytics Dashboard

This system enables the administrators to manage platform-wide operations, maintain quality control, and track user and transaction data through integrated analytics.

1.3.3 Module 03: AI Services (Flask + TensorFlow)

This subsystem represents the **intelligent assistance layer**, responsible for creativity enhancement, visual rendering, and predictive design functionalities. These services are implemented using Python-based frameworks and can be deployed independently for scalability and GPU optimization.

1.3.3.1 AI and Creative Intelligence Module

This module provides AI-based functionalities to enhance design workflows and user creativity. It features include:

1. 2D Preview Generator
2. 3D Renderer
3. Design Suggestion/ Generation Engine
4. Generative Canvas Enhancer

1.3.4 Module 04: Data Management Services (MongoDB-tentative)

This subsystem is dedicated to storing, organizing, and securing all persistent data across the platform. It ensures high availability, schema integrity, and fast access to stored entities.

1.3.4.1 Data Management Module

This module is responsible for structured data storage, schema design, and secure data access.

1. Collection Creation
2. Collection Management
3. Security Enforcement
4. Data Interface and Configuration Management

1.3.5 Module 05: Core Server Backend and Financial Operations (Node.js + Express + Payment Gateway)

This is the **central orchestration layer** of the system that manages communication between frontends, AI services, and the database. It also handles all business logic, financial workflows, and authentication.

1.3.5.1 Core Business Logic Module

1. Product Service
2. Order Service
3. Payment Service
4. Authentication Service
5. Messaging Service
6. SEO Service

1.3.5.2 Financial Operations Module

Handles automated, secure, and transparent financial transactions.

1. Split Payments
2. Designer Payouts
3. Manufacturer Payouts
4. Premium Billing Plans

1.3.6 System Overview

In summary:

- **Client Applications (Web)** handle all user interactions.
- **Admin Applications** provide system management and moderation.
- **AI Services** deliver intelligent design assistance, rendering, and personalization.
- **Data Management Services** ensure secure, reliable data persistence.
- **Core Server Backend** integrates all layers and manages payments, business logic, and APIs.

This architecture ensures **scalability**, **modularity**, and **separation of concerns**, allowing each system type to evolve independently while maintaining seamless interoperability. It also defines boundaries in the systems and the dependencies that may cross those boundaries.[2]

1.4 User Classes and Characteristics

The Orli platform is designed to serve multiple distinct user groups, each with its own objectives, system access levels, and behavioral patterns. These user classes differ from each other based on their privileges, frequency of interaction, and technical expertise.

Customers use the system primarily for browsing, customizing, and purchasing products. Designers act as creative content producers, managing storefronts and leveraging AI for design generation. Manufacturers operate as fulfillment partners, responsible for printing, packaging, and shipping. Administrators oversee user management, moderation, and platform analytics.

Additionally, system-level entities such as the AI Service and Payment Gateway operate as automated agents, facilitating creative intelligence and secure financial transactions, respectively. Developers and maintenance engineers support the system's operation, ensuring continuous delivery, scalability, and reliability across all modules.

Below is a comprehensive classification of user classes and their pertinent characteristics.

1.4.1 Customer (End User/ Buyer)

1.4.1.1 Description

Customers are general users who may browse, customize, and purchase products through Orli's web application. They shall interact primarily with the Customer UI module.

1.4.1.2 Characteristics

Customers:

- are non-technical, casual users with minimal training requirements - the typical online shopper.
- may access the platform through web or mobile browsers
- are focused on user experience, speed, and personalization
- may require visual feedback (product previews, recommendations)
- are security-conscious during payment and data entry
- may expect real-time order tracking and communication

1.4.1.3 Primary Goals

Customers' primary goals may be to:

1. browse and search for products and designs
2. preview items
3. place orders and make payments securely
4. track order and shipment status
5. review purchased product

1.4.2 Designer (Content Creator/ Seller)

1.4.2.1 Description

Designers are creative professionals who may create, upload, and sell their designs through Orli. They shall interact with both the Designer UI and AI Creative Intelligence modules.

1.4.2.2 Characteristics

Designers:

- are technically comfortable with design tools and digital assets (ideal case)
- may be comfortable with software environments; may be able to traverse dashboards
- may have an entrepreneurial mindset; will use platform to monetize creative work
- may require advanced interfaces (canvas editor, AI-assisted designing)
- may expect analytics, trend insights, and SEO control for storefronts
- may depend on transparency for sales and payout tracking.

1.4.2.3 Primary Goals

They may:

1. create or upload product designs
2. enhance designs using AI-assisted tools
3. manage storefront and product listings
4. view analytics (sales, engagement, customer trends)
5. accept and forward customer orders
6. receive revenue payouts

1.4.3 Manufacturer (Production Partner/ Vendor)

1.4.3.1 Description

Manufacturers are registered production partners who handle printing, packaging, and shipping of products. They may interact primarily with the Manufacturer UI.

1.4.3.2 Characteristics

Manufacturers:

- may be familiar with order management tools and dashboards
- may require accuracy and reliability in order details
- may need quick access to inventory data and shipping updates
- may be sensitive to production timelines and quality control

1.4.3.3 Primary Goals

Manufacturers may:

1. may receive and accept new production orders
2. may manage inventory and supply levels and update dashboard accordingly
3. may update order fulfillment and shipment status
4. may monitor performance metrics and analytics

1.4.4 Administrator (System Operator/ Platform Manager)

1.4.4.1 Description

Administrators shall oversee the entire Orli ecosystem, managing users, content, transactions, and platform health. They may use the Admin Web Application and Analytics Dashboard for this purpose.

1.4.4.2 Characteristics

Administrators shall be technically skilled and trained in operational or IT management (analytics, data management, auditing, dashboard management, etc.) and will require a secure, detailed, and data-rich interface. They will be responsible for enforcing policy compliance and content moderation. For this purpose, they will need system-wide visibility (users, payments, performance). They may also handle exceptions (disputes, fraud detection, policy breaches).

1.4.4.3 Primary Goals

Their main goals would be to:

1. manage and verify user accounts (designers, manufacturers, customers)
2. moderate content and flag inappropriate materials or respond to reports
3. monitor financial transactions and resolve issues
4. view system analytics (KPIs, uptime, engagement, revenue, etc.)

Table 1.1: User Classes, Interfaces, and System Interaction Characteristics for Orli

User Class	Access Level	Main Interface	Interaction with System	Impact
Customer	Basic	• Web App • Customer UI	• Browse and search products • Customize and preview designs • Place and pay for orders • Track shipments and view trends	• System on User: Simplifies product discovery and personalization • User on System: Generates behavior data influencing recommendations
Designer	Medium	• Designer Web Dashboard • AI Canvas Tools	• Create and manage product designs • Enhance designs using AI • Manage storefront and analytics	• System on User: Provides AI support and sales insights • User on System: Contributes to creative content and storefront data
Manufacturer	Medium	• Manufacturer Dashboard • Inventory Panel	• Accept and fulfill orders • Update shipment and stock status	• System on User: Automates workflow and order tracking • User on System: Ensures timely delivery and production accuracy, ensuring platform success

Continued on next page

User Class	Access Level	Main Interface	Interaction with System	Impact
Administrator	High	• Admin Portal • Analytics Dashboard	• Manage users and permissions • Moderate content and monitor system performance	• System on User: Offers visibility and control over the platform • User on System: Maintains platform integrity and policy compliance

In addition, some external system components and maintenance team also exists that will be interacting with the system even after the launch in order to keep the system functioning, like payment APIs, AI-service, database provider, hosting service etc. While they might not fall directly into the user category, they still significantly influence the system and frequently interact with it.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-cases

Applying the use-case discovery framework as explained by **Craig Larman** in his book **Applying UML and Patterns**, we started by defining the system boundaries, finding relevant actor with explicit goals and validated use-cases. As mentioned before, Orli is a web-based print-on-demand platform that connects designers, manufacturers, and customers through an AI-enhanced marketplace.

2.1.1 Identifying System Boundaries, Actors and their Goals

Orli includes:

1. A Client Web Application (for Customers, Designers, Manufacturers)
2. An Admin Web Application (for platform management)
3. Backend services (Business Logic, AI Services, Payment, Data Management)

Inside the Boundary (of the system) we have user interfaces, core backend APIs (products, orders, payments, auth), AI modules for design generation and rendering, marketplace features (storefronts, analytics, checkout), payment and fulfillment workflows. Outside the system, we have third-party payment gateways, external shipping services(business end), hosting platform, and our model training infrastructure. Furthermore, each separate component has its own boundaries that will define the architecture of the system.

Secondly, the actor in Orli would be the end-user i.e. designers, manufacturers, customers and administrators. External system components like the AI-engine and the payment services will also

be considered, but they will come under the class of external actors or system actors. The details of refining use-cases are as follows:

Table 2.1: Actor Summary

Actor	Role in the System	High-Level Goals
Customer	End-user who buys customized products	• They may browse, customize, and purchase unique merchandise easily. • They will engage with product previews and recommendation system only
Designer	Artist or entrepreneur creating and selling designs	• They may create and upload product designs. • They may manage their storefronts and pricing. • They may track performance and earnings.
Manufacturer	Production partner responsible for printing and shipping orders	• They may receive and process incoming print orders. • They may manage production queues and shipment schedules.
Admin	Platform owner or moderator	• They may oversee user activity and content quality. • They will ensure policy compliance and maintain platform integrity.
AI Service (System Actor, Secondary Actor)	Intelligent assistant within the system	• It may generate or enhance design elements based on user input. • It may assist designers in other actions such as writing descriptions, etc.
Payment Gateway (External Actor, Secondary Actor)	Third-party financial system	• It may process transactions and payouts securely. • It will ensure compliance with payment and data standards.

Table 2.2: Actor–Goal (Use Case) Mapping and Validation for Orli

Actor	User Goal (Use Case Name)	EBP / Boss / Size Test Result
Customer	Browse and search products	Passes Boss Test — adds business value, involves filtering, searching, previewing. (discover products).
	Place and pay for an order	Major business transaction.
	Manage account/profile	Slightly low-value but necessary for usability.
Designer	Create new designs	Core creative process; measurable contribution.
	Enhance or generate design using AI	AI-assisted workflow, passes EBP Test.
	Set up and manage storefront	Adds measurable business value through sales, complex workflow.
	View and manage sales analytics	Analytical EBP; supports business decisions, but just reviewing use-case, therefore ignored.
Manufacturer	Receive and fulfill orders	Business-end process, involve, status updation, inventory management.
	View performance reports	Strategic analysis; value-adding process but too small, therefore ignored.
Admin	Manage platform users (add/remove)	Moderation EBP; adds system integrity, but single step workflow, therefore, ignored.
	Monitor system analytics	Supervisory EBP; valuable operational insight, single step usecase in use case format, therefore, not included.

Continued on next page

2. Project Requirements

Actor	User Goal (Use Case Name)	EBP / Boss / Size Test Result
	Moderate reported content	Compliance EBP; maintains quality and trust, complex process that may/may not include removal of users/ communication designs, making policies etc., business end process

This brings us to a concise list of identified use-cases:

1. Browse/Search Products
2. Place Order
3. Manage Account (generic and for all users in every platform, therefore not considered)
4. Create Design
5. Enhance/Generate Design using AI
6. Manage Storefront
7. Moderate Platform (includes, users, content, etc.)
8. Monitor Analytics

2.1.2 Use Cases

The extended use-case summary of Orli is as follows:

2.1.2.1 Use Case 1: Browse and Search Products

Use Case ID: UC-01

Use Case Name: Browse and Search Products

Primary Actor: Customer

Stakeholders and Interests:

- Customer: Wants to quickly find relevant products matching their interests
- Designer: Wants their products to be discoverable
- Platform: Wants to facilitate sales and user engagement

Preconditions:

- System is online and accessible
- Product catalog contains at least one published product

Postconditions (Success Guarantee):

- Customer views list of products matching search criteria
- Search history is logged for analytics
- Product views are recorded

Main Success Scenario:

1. Customer navigates to marketplace homepage
2. Customer enters search keywords in search bar

3. System validates input and queries product database
4. System applies default filters (available products, active designers)
5. System retrieves matching products with metadata
6. System displays results in grid layout with thumbnails, names, prices
7. Customer applies additional filters (price range, category, designer)
8. System updates results dynamically without page reload
9. Customer selects sorting preference (relevance, popularity, newest)
10. System reorders results accordingly
11. Customer clicks on a product to view details
12. System displays detailed product page (triggers UC-02)

Extensions (Alternate Flows): *3a. Invalid input detected:*

- System sanitizes input to prevent injection attacks
- System displays sanitized results

4a. No products match search criteria:

- System displays “No results found” message
- System suggests popular products or trending designs
- Customer can modify search or browse categories

7a. Applied filters yield no results:

- System notifies customer that no products match all filters
- System suggests removing some filters
- Customer adjusts filters or clears them

11a. Product becomes unavailable during browsing:

- System displays “Product no longer available” message
- System suggests similar products

Special Requirements:

- Search results must load within 2 seconds on standard connection
- Interface must be responsive for mobile and desktop
- Search must support Urdu keywords for Pakistani market

Technology and Data Variations:

- Search can use Elasticsearch or MongoDB text indexes
- Results can be cached for popular queries

Frequency of Occurrence: High (estimated 1000+ times per day)

Open Issues:

- Should voice search be supported?
- Implement fuzzy matching for typos?

2.1.2.2 Use Case 2: Place Order

Use Case ID: UC-02

Use Case Name: Place and Pay for Order

Primary Actor: Customer

Stakeholders and Interests:

- Customer: Wants secure, fast checkout experience
- Designer: Wants order confirmation to start earning
- Manufacturer: Wants clear production orders
- Payment Gateway: Wants secure transaction processing
- Platform: Wants commission from successful sales

Preconditions:

- Customer is logged in with verified account
- Customer has items in shopping cart
- Selected products are in stock
- Payment gateway service is operational

Postconditions (Success Guarantee):

- Order is created with unique ID in database
- Payment is processed and confirmed
- Funds are split between designer, manufacturer, and platform
- Designer and manufacturer are notified of new order
- Customer receives order confirmation
- Inventory is reserved for order

Main Success Scenario:

1. Customer reviews shopping cart with selected items
2. Customer clicks “Proceed to Checkout”
3. System validates cart items availability
4. System calculates subtotal, taxes, shipping charges
5. System displays checkout page with total amount
6. Customer enters or confirms shipping address
7. System validates address format
8. Customer selects payment method (card/wallet/bank)

9. Customer enters payment credentials
10. System validates payment information format
11. System sends payment request to Payment Gateway API
12. Payment Gateway authenticates and authorizes transaction
13. Payment Gateway returns success response with transaction ID
14. System creates order record in database
15. System executes split payment logic (designer %, manufacturer %, platform %)
16. System updates inventory status
17. System sends notification to designer with order details
18. System sends notification to manufacturer with production specs
19. System sends confirmation email to customer with order ID
20. System displays order summary page to customer

Extensions (Alternate Flows): *3a. One or more cart items unavailable:*

- System identifies unavailable items
- System notifies customer which items cannot be ordered
- System removes unavailable items from cart
- Customer decides to continue or modify cart

7a. Invalid shipping address:

- System highlights errors in address fields
- System suggests corrections (if address validation API available)
- Customer corrects address
- Return to step 7

12a. Payment declined by gateway:

- Payment Gateway returns error code and reason
- System logs failed transaction attempt
- System creates order with “Payment Failed” status
- System displays error message to customer
- System offers to retry or use alternate payment method
- Customer retries or cancels order

12b. Payment gateway timeout:

- System waits for 30 seconds maximum
- System logs timeout event
- System displays “Payment status unclear” message

- System initiates payment verification query
- If verified as paid, proceed to step 14
- If verified as failed, proceed to 12a
- If still unclear, customer service intervention required

15a. Split payment calculation error:

- System logs calculation error
- System holds full payment amount
- System notifies admin of manual review needed
- Admin resolves and manually triggers split

Special Requirements:

- Payment data must be encrypted using TLS 1.3
- PCI DSS compliance required for card payments
- System must support Pakistani payment methods (JazzCash, EasyPaiza)
- Order confirmation must be generated within 5 seconds of payment success

Technology and Data Variations:

- Payment gateway can be Stripe, JazzCash, or EasyPaiza
- Order data stored in MongoDB with transaction logs
- Notification can be email, SMS, or in-app

Frequency of Occurrence: Medium-High (estimated 50–200 orders per day)

Open Issues:

- Should we support cash on delivery?
- What happens if notification services fail after successful payment?
- Define refund policy and implementation

2.1.2.3 Use Case 3: Create and Enhance Design Using AI

Use Case ID: UC-03

Primary Actor: Designer

Stakeholders and Interests:

- Designer: Wants easy-to-use tools for professional designs
- AI Service: Provides accurate enhancements
- Customer: Wants high-quality designs
- Platform: Wants diverse, quality content

Preconditions:

- Designer logged in
- Design canvas accessible
- AI service operational

Postconditions:

- Design enhanced and saved
- File stored in secure storage
- Logged for analytics

Main Success Scenario:

1. Designer clicks “Create New Design”
2. System opens design canvas
3. Designer uploads image
4. System validates format
5. Image uploaded to cloud
6. Designer edits and enhances via AI
7. AI returns enhanced version
8. Designer approves and saves

Extensions:

- Invalid file format or size
- Autosave or AI failure
- Designer rejects enhanced version

Special Requirements:

- Undo/redo (20 steps)
- AI enhancement <30s
- Min 300 DPI

2.1.2.4 Use Case 4: Manage Storefront

Use Case ID: UC-04

Primary Actor: Designer

Stakeholders:

- Designer, Customer, Platform, SEO

Main Success Scenario:

1. Designer edits storefront info, images, and products
2. System validates fields

3. Designer optimizes SEO
4. System updates and publishes

Extensions:

- Validation errors or cache failures handled gracefully

Special Requirements:

- Unique URLs
- SEO title < 60 chars
- Meta description 150–160 chars

2.1.2.5 Use Case 5: Fulfill Order (Manufacturer)

Use Case ID: UC-05

Primary Actor: Manufacturer

Main Success Scenario:

1. Manufacturer reviews and accepts orders
2. Begins production and updates status
3. Marks as shipped and provides tracking ID

Extensions:

- Material shortage
- Quality check fails
- Invalid tracking ID

Special Requirements:

- Print-ready files (CMYK, 300+ DPI)
- Must update within 24 hours

2.1.2.6 Use Case 6: Moderate Platform Content

Use Case ID: UC-06

Primary Actor: Administrator

Stakeholders:

- Administrator, Designers, Customers, Platform

Main Success Scenario:

1. Admin logs in and reviews moderation queue
2. Reviews flagged content and applies policy
3. Takes action (remove, warn, suspend)
4. Logs decision and notifies affected users

Postconditions:

- Violations handled and logged
- Users notified

The last two use cases are a one step black box use cases and may be entirely revamped on development, therefore the success scenarios and main workflow is unidentified as of now.

2.1.3 UML Use Case Diagram

The use-case diagram is essentially, the use-cases in Unified Modeling Language. In UML, Orli looks something like this:

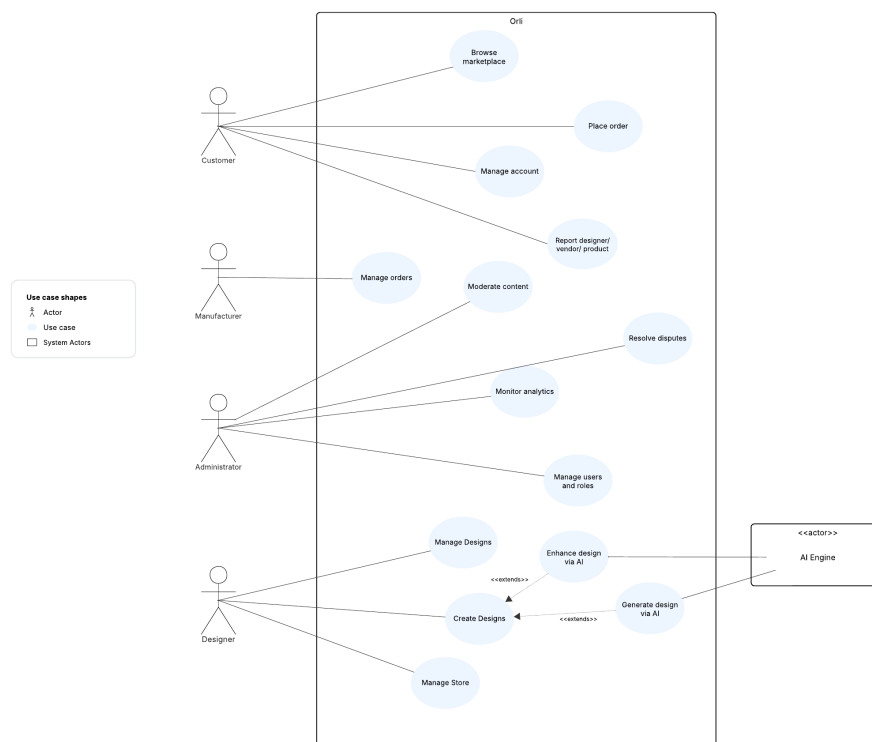


Figure 2.1: UML Use-Case Diagram

The above diagram, forgoes any obvious prerequisites like login, signup, authorizations user management, etc, since they don't necessarily qualify as meaningful tasks or use-cases according to

any of the use-case validation tests. Moreover, they are also same as everyother platform therefore do not qualify as valid use case nor add business value to the sytem.

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style.

2.2.1 Module 1: Client Web Application (React Frontend)

2.2.1.1 Customer Interface

Feature 01: Browse and Search Products

1. The system shall allow customers to view all available products with thumbnails, names, and prices.
2. The system shall allow customers to search for products using keywords, categories, or tags.
3. The system shall display search results dynamically without reloading the page.
4. The system shall provide filters (price range, category, designer, etc.) to refine search results.
5. The system shall sort products by relevance, popularity, or newest first.
6. If no results match the query, the system shall display an appropriate message.

Feature 02: Preview Product

1. The system shall allow customers to select a product and open a detailed view page.
2. The system shall allow customers to preview product variants (e.g., color, size, design).
3. The system shall provide a 3D rendering of the customized product using design preview API.
4. The system shall notify the user if the product is out of stock.

Feature 03: Place and Pay for Orders

1. The system shall allow customers to add custom products to a shopping cart.

2. The system shall calculate total cost, taxes, and shipping charges automatically.
3. The system shall allow customers to select payment methods (credit/debit card, wallet, etc.).
4. The system shall validate payment information before processing.
5. The system shall include the “Validate Payment” function to ensure secure transaction verification.
6. The system shall confirm order placement and display a summary page after successful payment.
7. The system shall invoke the “Handle Payment Failure” use case if payment is declined or timed out.

Feature 04: Track Order Status

1. The system shall allow users to view the status of all their past and current orders.
2. The system shall display real-time updates (Processing, Shipped, Delivered, Canceled).
3. The system shall include shipment tracking links where available.
4. The system shall notify users automatically when order status changes.

Feature 05: View Personalized Recommendations

1. The system shall display AI-driven product suggestions based on browsing and purchase history.
2. The system shall update recommendations dynamically as user preferences change.
3. The system shall ensure user privacy when using stored data for recommendations.

2.2.1.2 Designer Interface

Feature 06: Design Canvas (Adobe-like Features)

1. The system shall provide a web-based design workspace with basic drawing, layering, and text editing tools.
2. The system shall support file uploads in PNG, JPEG, and SVG formats.
3. The system shall allow undo/redo actions and autosave progress.
4. The system shall handle upload failures via “Handle Design Upload Failure.”

Feature 07: Storefront Creation and Management

1. The system shall allow designers to create a personal storefront with name, logo, and description.
2. The system shall allow designers to add or remove products from their storefront.
3. The system shall provide SEO fields for product title, description, and tags.
4. The system shall allow designers to update pricing. (tentative)
5. The system shall notify designers upon successful storefront updates.

Feature 08: Accept Orders

1. The system shall allow designers to view new customer orders and their details.
2. The system shall allow designers to accept or reject custom order requests.
3. The system shall automatically notify customers when an order is accepted or rejected.

Feature 09: Trend Analytics

1. The system shall display analytics for designer storefronts, such as sales, visitors, and most popular items.
2. The system shall generate visual reports for designer performance (includes “Generate Reports”).
3. The system shall notify designers of significant sales milestones achievements.

Feature 10: Product Page SEO Optimization

1. The system shall suggest SEO-friendly titles and tags for designer products using AI analysis or dictionary. (tentative)
2. The system shall allow manual keyword editing and provide real-time feedback.

2.2.1.3 Manufacturer Interface**Feature 11: Accept Orders**

1. The system shall display a queue of orders assigned to the manufacturer.
2. The system shall allow manufacturers to view product details, quantity, and deadlines.
3. The system shall allow acceptance or rejection of incoming orders.
4. The system shall log all manufacturer decisions for auditing.

Feature 12: Print and Ship

1. The system shall allow manufacturers to mark orders as printed or shipped.
2. The system shall allow entry of tracking IDs and shipment dates.
3. The system shall automatically trigger notifications to customers upon shipping.
4. The system shall include “Handle Order Fulfillment Delay” when manufacturing issues occur.

Feature 13: Inventory Management

1. The system shall display the available inventory list and current stock levels.
2. The system shall allow adding, editing, or removing inventory items.
3. The system shall alert manufacturers when stock levels fall below predefined thresholds.
4. The system shall extend to “Handle Inventory Shortage” when a requested item is unavailable.

Feature 14: Performance Statistics

1. The system shall display metrics such as average fulfillment time, total orders processed, and defect rates.
2. The system shall allow exporting performance data for administrative reporting.

2.2.1.4 Admin Interface

Feature 15: User Management

1. The system shall allow the admin to view all registered users.
2. The system shall allow adding, editing, or deactivating user accounts.
3. The system shall restrict sensitive actions to users with admin privileges.
4. The system shall include “Handle Unauthorized Access” for privilege violations.

Feature 16: System Monitoring

1. The system shall display key performance indicators such as uptime, API response time, and transaction volume.
2. The system shall alert the admin of anomalies such as server downtime or failed transactions.

Feature 17: Content Moderation

1. The system shall allow admins to review and remove inappropriate designs or comments.
2. The system shall include “Report Inappropriate Content” triggered by users.
3. The system shall maintain a moderation history log.

Feature 18: Analytics Dashboard

1. The system shall aggregate and visualize platform-wide metrics (sales, traffic, new users).
2. The system shall generate downloadable reports.
3. The system shall include “Handle Report Generation Failure” if data fetching fails.

2.2.2 Module 2: AI and Creative Intelligence (Flask + TensorFlow)**Feature 01: 2D Preview Generator**

1. The module shall generate flat 2D mockups of designs on selected merchandise templates.
2. The module shall process designer inputs within 5 seconds for standard resolutions. (still under development, therefore tentative KPI)
3. The module shall notify the frontend if rendering fails.

Feature 02: 3D Renderer

1. The module shall render 3D previews of products using uploaded or generated designs.
2. The module shall allow rotation, zoom, and lighting effects for user visualization.
3. The module shall include “Handle Rendering Failure” for GPU or timeout errors.

Feature 03: Design Suggestion / Generation Engine

1. The module shall generate design ideas based on user prompts or uploaded sketches.
2. The module shall ensure generated designs comply with ethical and copyright constraints.
3. The module shall invoke “Handle AI Generation Failure” when inference fails.

Feature 04: Generative Canvas Enhancer

1. The module shall enhance low-resolution designs using up-scaling or inpainting AI.
2. The module shall log all enhancement activities for future retraining or audits.(architecture under consideration, therefore tentative)

2.2.3 Module 3: Core Business Logic (Node.js + Express)

Feature 01: Product Service

1. The service shall manage product creation, updates, and deletion.
2. The service shall validate inputs before saving to the database.

Feature 02: Order Service

1. The service shall orchestrate order creation, tracking, and updates.
2. The service shall notify relevant actors (customer, designer, manufacturer) when order state changes.

Feature 03: Payment Service

1. The service shall manage payment requests, responses, and logs.
2. The service shall communicate securely with the Payment Gateway API.
3. The service shall log all failed transactions for audit and debugging.

Feature 04: Auth Service

1. The service shall authenticate and authorize all API calls.
2. The service shall include “Handle Unauthorized Access” for invalid credentials.

Feature 05: Messaging Service

1. The service shall handle user-to-user and system-to-user communications.
2. The service shall ensure message delivery even under partial failure conditions.

Feature 06: SEO Service

1. The service shall optimize URLs and metadata for better search visibility.

2.2.4 Module 4: Data Management (MongoDB)

1. The module shall create and manage collections for users, products, orders, payments, and logs.
2. The module shall enforce data validation rules and schema consistency.
3. The module shall ensure secure read/write operations using role-based permissions.
4. The module shall handle “Data Sync Failures” by retrying transactions and maintaining logs.

2.2.5 Module 5: Financial Operations (Payment Gateway Integration)

Feature 01: Split Payments

1. The module shall automatically divide payment amounts between designer, manufacturer, and platform commission.

Feature 02: Designer Payouts

1. The module shall initiate scheduled payouts to designer accounts upon reaching the threshold.
2. The module shall handle failed payout retries automatically.

Feature 03: Manufacturer Payouts

1. The module shall calculate and issue payments to manufacturers based on fulfilled orders.
2. The module shall notify the admin upon any transaction errors.

Feature 04: Premium Billing Plan

1. The module shall allow designers to upgrade to premium features.
2. The module shall manage recurring billing and notify users of upcoming renewals.

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements of Orli.

2.3.1 Reliability

- All critical operations (payments, order creation) shall have **automatic retry mechanisms** for failed transactions.
- The system shall store all submitted designs and orders redundantly to prevent loss in case of server restarts.

2.3.2 Usability

- The user interface shall maintain consistent layout and color themes across all pages
- Users shall be able to navigate from homepage to checkout in **no more than 4 clicks**
- Error messages shall be displayed in clear, human-readable language (no system codes)

2.3.3 Performance

- The system shall load the main marketplace page within **5 seconds** on a standard 4G connection
- The system shall handle at least **50 concurrent users** without visible slowdown or request time-out
- The AI-based 2D preview generator shall produce results within **30 seconds**.

2.3.4 Security

- The system shall ensure that cross site scripting will not affect it.
- The system shall ensure that all external integrations, such as AI services or payment gateways, operate over secure, authenticated channels.
- The system shall take measures to prevent SQL injections
- The system shall ensure that all user authentication and authorization processes are performed securely and that only authorized users can access restricted resources.
- The system shall detect and respond to repeated failed login attempts or other suspicious activities in a timely manner.

2.4 Domain Model

After all the above deliberation, the abstract shape of Orli looks something like this:

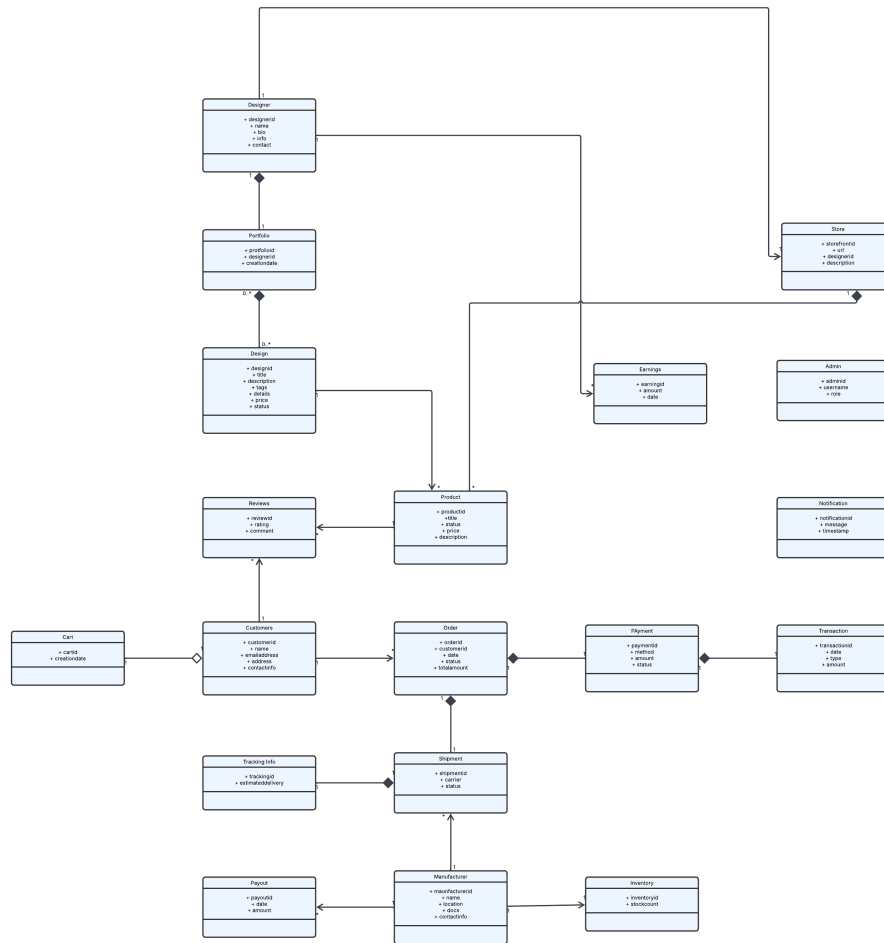


Figure 2.2: UML Domain Model

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

Orli has been developed using a modular, layered, and service-oriented architecture following the principles of Clean Architecture and Separation of Concerns. It follows the **onion architecture** view from **Head First Design Patterns**.^[1] The system is decomposed into distinct modules, each responsible for a specific set of functionalities. The system, as a whole, follows a multi-layered client-server model, extended with AI integration services and data management subsystems. This decomposition was done based on two concerns:

1. We wanted the core business rules (the "what") independent of the implementation i.e. how it happens.
2. We wanted the modules to evolve or be replaced (e.g., database or AI engine) without impacting the rest of the system.

Hence, the system is divided into five primary modules:

1. client application (presentation layer)
2. core business logic (domain layer)
3. API layer (service layer)
4. creative intelligence layer
5. data management layer

The AI layer and Data Layer are shown as single entity because both come under the guise of persistent storage and store valuable data; one stores user dat and other stores AI assets.Each module operates independently but collaborates via well-defined interfaces or APIs. This ensures that the application layer mediates all communication and the core logic remains isolated and intact at all times. Moreover, the interfaces never impact the system implementation.

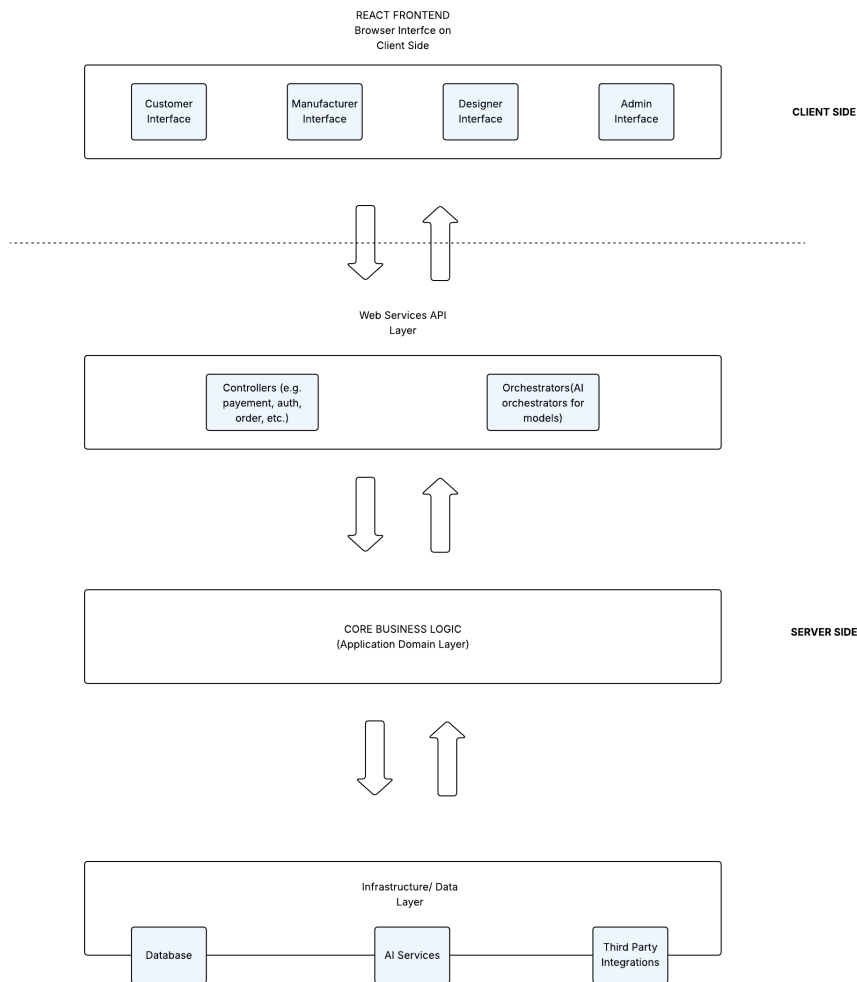


Figure 3.1: System Architecture

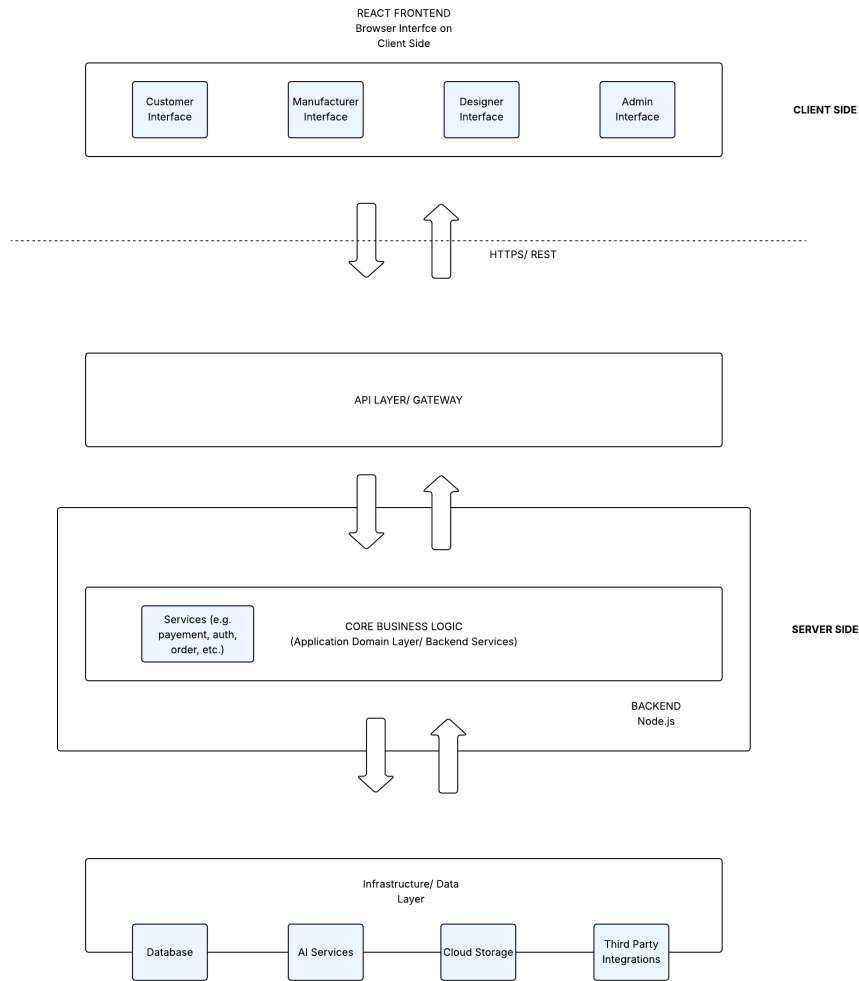


Figure 3.2: System Architecture from another View

3.2 Design Models

The system architecture is supported by the following diagrams including class diagram, sequence diagram, system sequence diagrams, and activity diagrams. They elaborate on the use cases and what the system will do. Since major actions and workflows involve more than on use cases, and not stand alone use cases, therefore the diagrams may utilize multiple workflows.

3. System Overview

UML Class Diagram of DSA
Akhil Aul / October 12, 2023

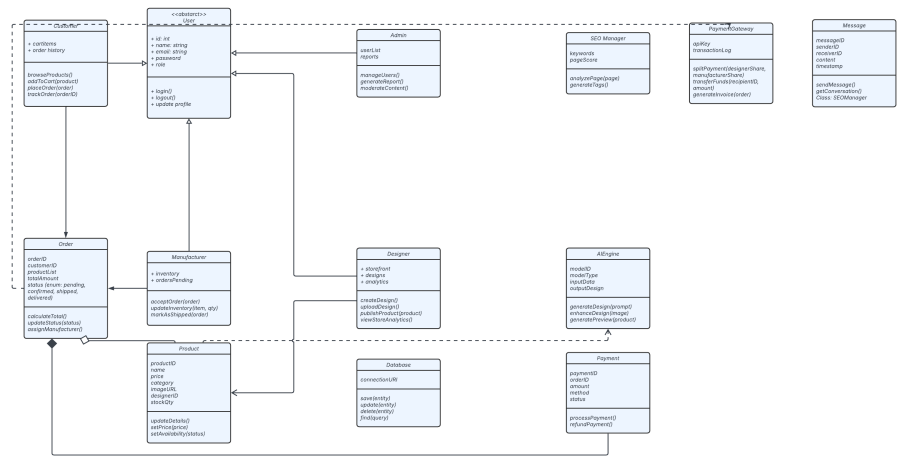


Figure 3.3: Class Diagram

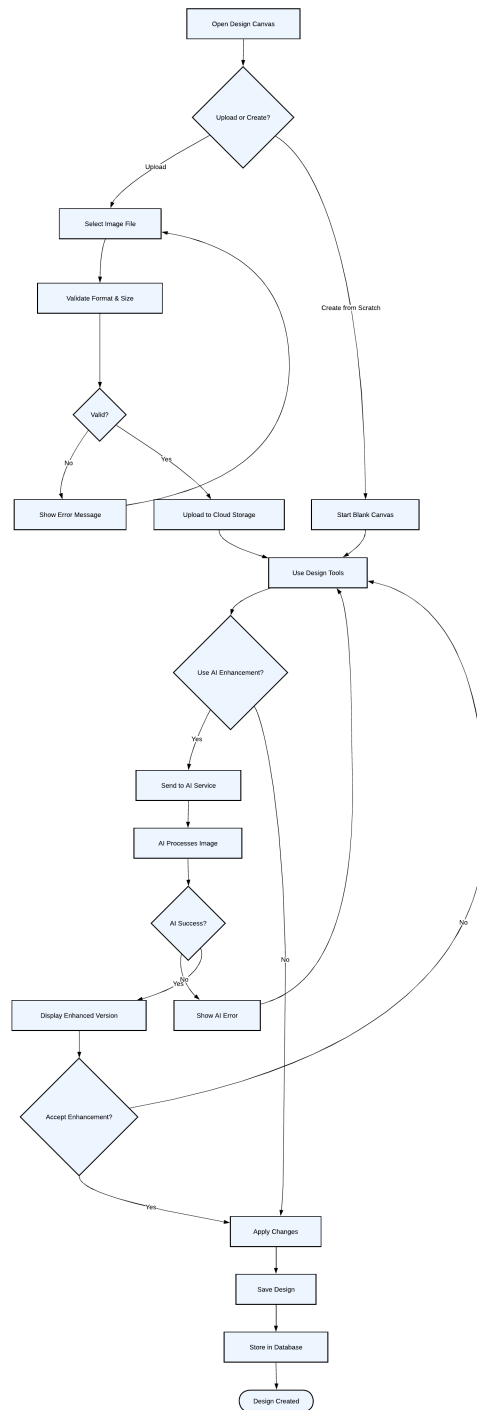


Figure 3.4: Activity Diagram for Designers

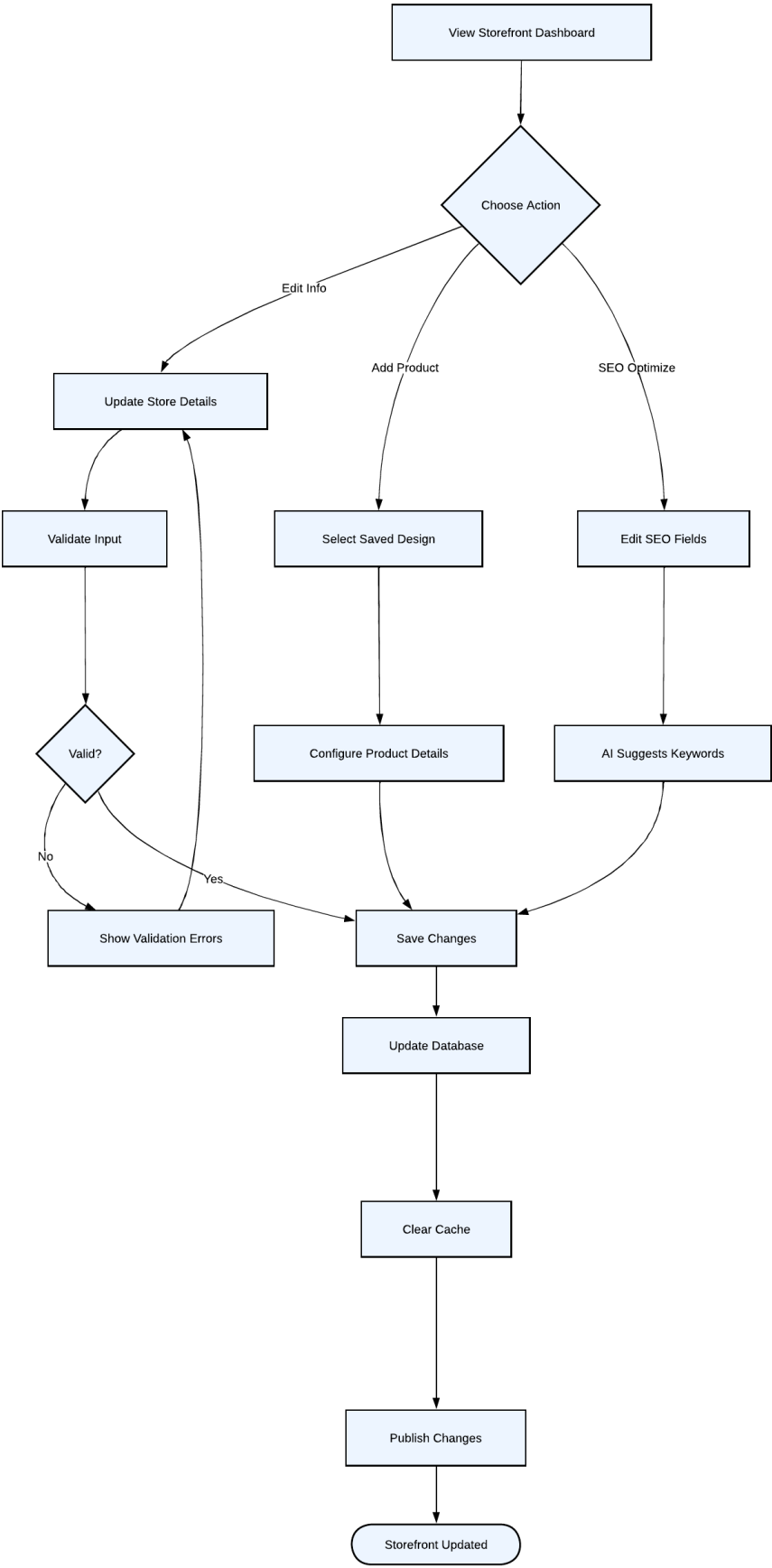


Figure 3.5: Activity Diagram for Designers

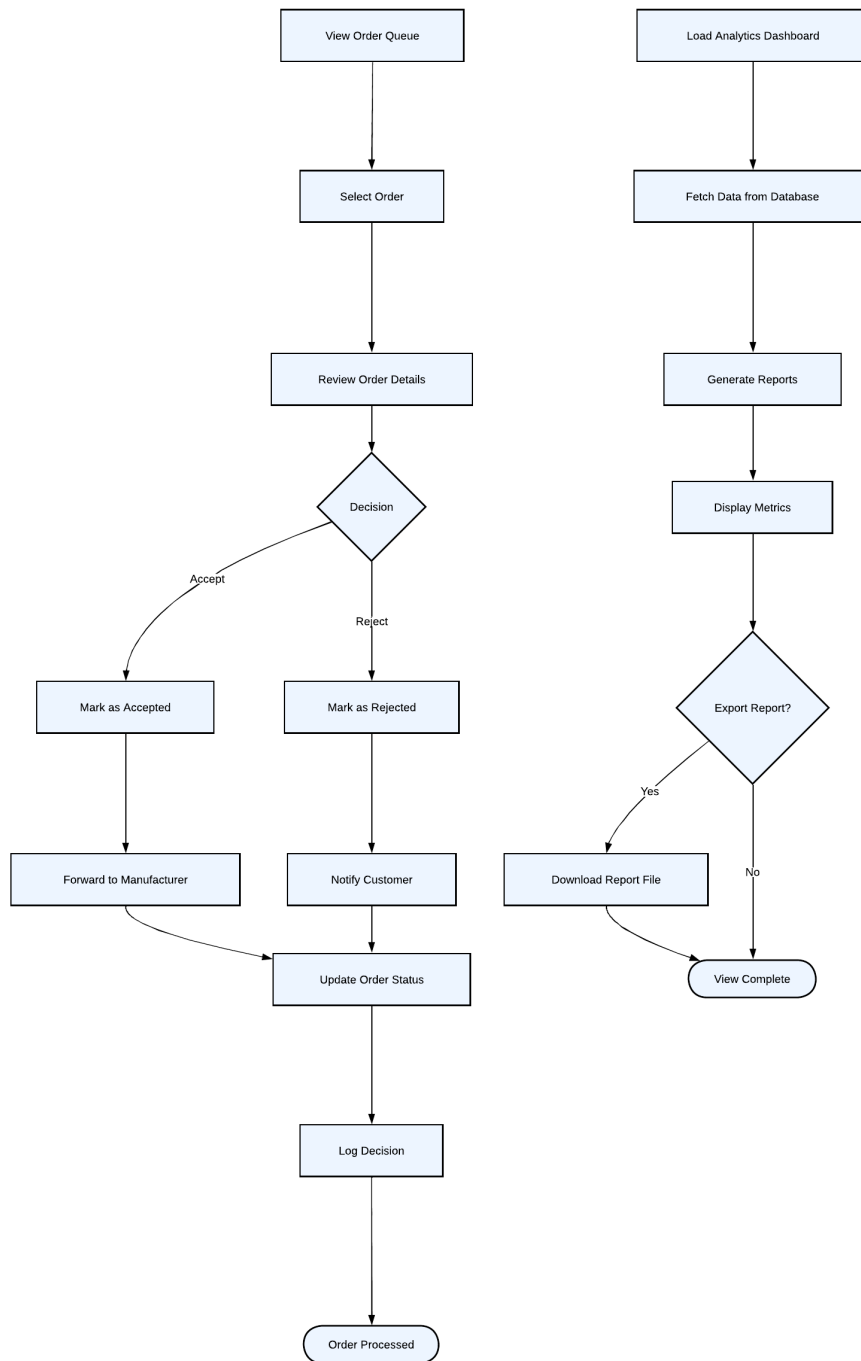


Figure 3.6: Activity Diagram for Designers

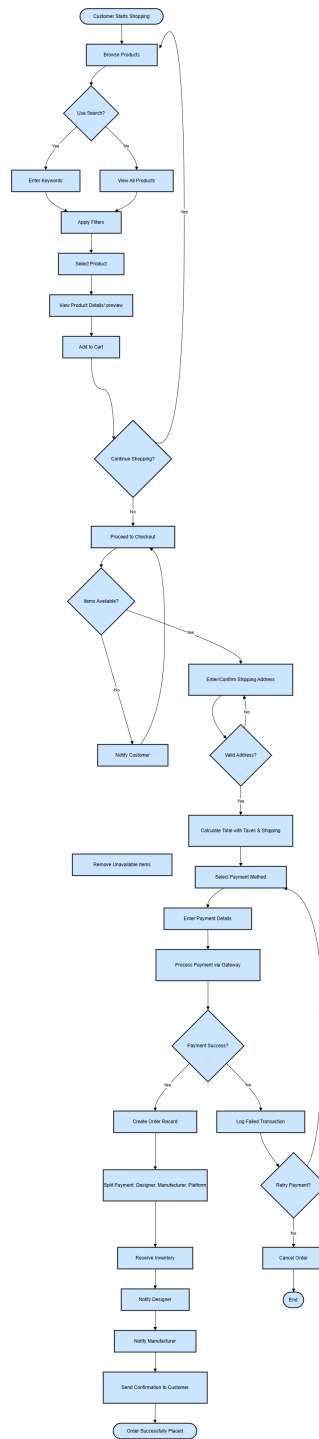


Figure 3.7: Activity Diagrams for Customers

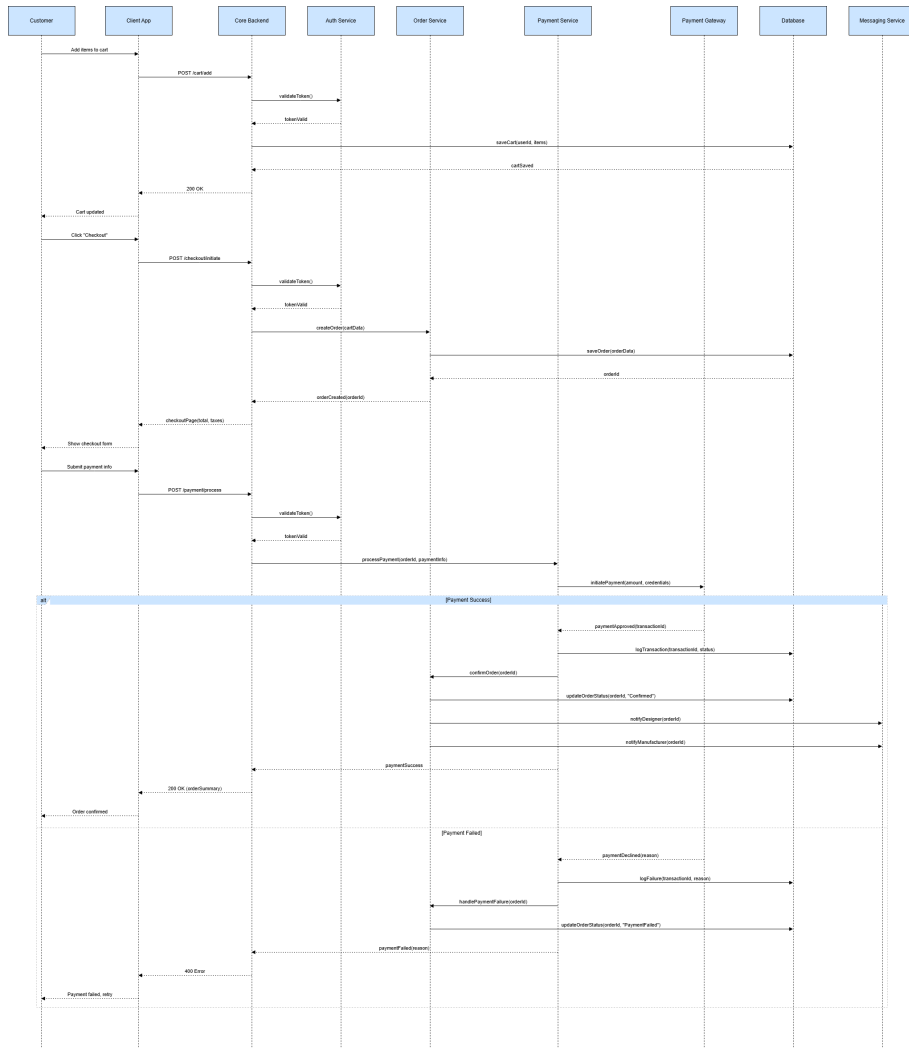


Figure 3.8: Sequence Diagram for Customer for Order Placement

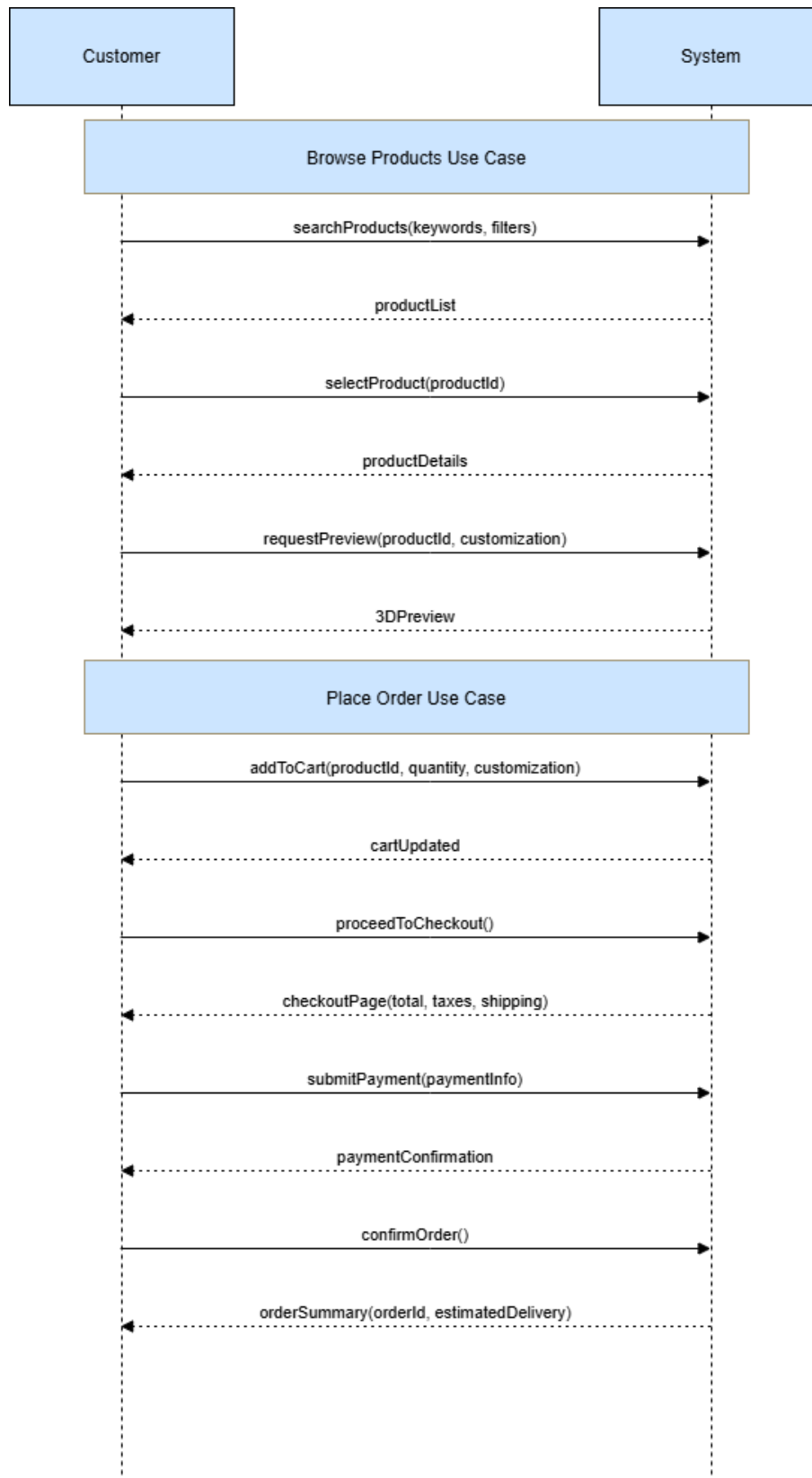


Figure 3.9: System Sequence Diagram for Order Placement

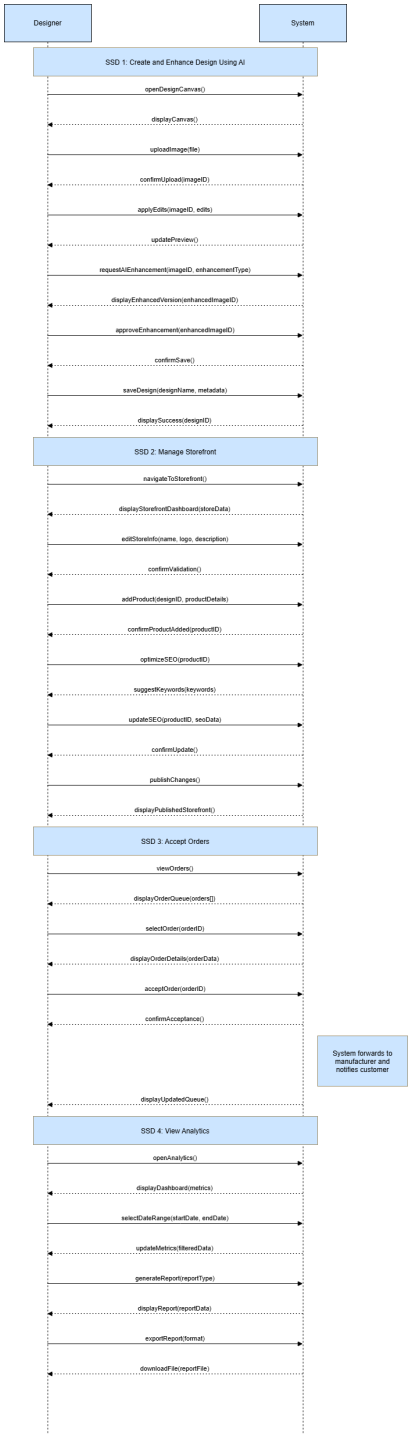


Figure 3.10: System Sequence Diagram for Designer Usecases

3.3 Data Design

The Orli platform transforms its information domain into structured data entities stored primarily in **MongoDB**, tentatively, following a domain-driven design approach. Each business entity such

as *User*, *Design*, *Product*, *Order*, *Storefront*, and *Analytics* directly maps to a database collection.

3.3.0.1 Data Transformation

- **Conceptual to Physical:** Business entities are identified from the domain model, converted into schemas with validation and indexes, and stored as documents for scalability and performance.

3.3.0.2 Major Entities

- **User:** Stores account details, roles (customer, designer, manufacturer, admin), and profile information.
- **Design:** Contains designer-created or AI-generated artwork with metadata and file URLs.
- **Product:** Represents merchandise items linked to designs and manufacturers, including SEO and analytics fields.
- **Order:** Records customer purchases, fulfillment details, and split payment data.
- **Storefront:** Holds designer shop information, themes, and featured products.
- **Analytics and AI Assets:** Capture user engagement metrics and AI processing outputs.
- **Design Assets:** Maintains asset library for Designers

3.3.0.3 Data Storage Systems

- **Database (e.g., MongoDB):** Main database for transactional and relational data.
- **Cloud Storage (e.g., AWS S3):** Stores images, design files, and previews; MongoDB holds only URLs.
- **Cache:** Temporarily stores frequently accessed data for faster performance.
- **AI Model Storage:** Keeps trained AI model versions for design generation.

3.3.0.4 Processing and Organization

Data flows through APIs where user actions (such as uploads or orders) trigger MongoDB updates. Media files are uploaded to cloud storage, while analytics and AI data are aggregated and stored asynchronously.

3.3.0.5 Summary

The Orli system organizes all information into well-structured MongoDB collections with supporting cloud and cache systems, ensuring scalable, secure, and efficient data storage and processing across the platform.

Bibliography

- [1] Craig Larman. *Applying UML and Patterns: An Introduction to Object Oriented Analysis and Design and the Unified Process, Second Edition*. Prentice Hall Professional, 2001.
- [2] Rober C. Martin. *Clean Architechure*. Pearson Education, Inc., 2018.